

TRON Foundation Library and Upgradeable Contracts Audit



August 21, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	12
Security Model and Trust Assumptions	13
Privileged Roles	13
Trust Assumptions	14
Additional Considerations	16
High Severity	17
H-01 TRC4626 And VestingWallet Cannot Release USDT Due To False-On-Success Transfers	17
H-02 Audited Contracts Inherit Issues From Upstream OpenZeppelin Contracts	18
Medium Severity	20
M-01 InputSettlerEscrowTron Uses Ethereum Permit2 Address, Breaking openFor on TRON	20
Low Severity	21
L-01 TRC20FlashMint Rejects TIP-3156-Compliant Flash Borrowers	21
L-02 Documentation Cites Ethereum ERC/EIP Names for Standards Renamed to TRON Identifiers	21
L-03 Missing Documentation for Decoding 21-Byte TRON Addresses	23
Notes & Additional Information	25
N-01 TRON Block Time Compresses Block-Denominated Governance Windows Roughly Fourfold	25
N-02 Stale Warnings And Cautions In The Codebase	26
N-03 Rename contracts from OpenZeppelin Contracts to Tron Contracts	27
N-04 Governance Executors Do Not Support Forwarding or Recovery of TRC-10 Tokens	27
N-05 README and Documentation Reference ERC/EIP-Named Artifacts That Only Exist Under TRC Names	28
Conclusion	30
Appendix	31
Issue Classification	31

Summary

Type	Library	Total Issues	11 (11 resolved)
Timeline	From 2026-07-02 To 2026-07-22	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	2 (2 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	3 (3 resolved)
		Notes & Additional Information	5 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited three repositories. The first two make up the TRON port of the OpenZeppelin Contracts library, and the third is the TRON adaptation of the Open Intents Framework settlement contracts:

- the [OpenZeppelin/tron-contracts](#) repository at commit [06d69bc](#),
- the [OpenZeppelin/tron-contracts-upgradeable](#) repository at commit [f66f953](#)
- the [openintentsframework/oif-contracts](#) repository at commit [39df288](#).

For the two OpenZeppelin libraries, in scope were all Solidity sources under the [contracts/](#) directory, excluding the mock and test helpers under [contracts/mocks/](#). The two libraries share the same module layout; the upgradeable repository provides the upgradeable variants of the same standards and therefore omits the proxy and interface modules that only the non-upgradeable library ships. For the Open Intents Framework repository, in scope were the TRON-specific input-settlement contracts. The complete in-scope file lists are given below.

The in-scope files of [tron-contracts](#) (216 files) were the following:

```
├── contracts
│   ├── access
│   │   ├── AccessControl.sol
│   │   ├── IAccessControl.sol
│   │   ├── Ownable.sol
│   │   ├── Ownable2Step.sol
│   │   ├── extensions
│   │   │   ├── AccessControlDefaultAdminRules.sol
│   │   │   ├── AccessControlEnumerable.sol
│   │   │   ├── IAccessControlDefaultAdminRules.sol
│   │   │   └── IAccessControlEnumerable.sol
│   │   └── manager
│   │       ├── AccessManaged.sol
│   │       ├── AccessManager.sol
│   │       ├── AuthorityUtils.sol
│   │       ├── IAccessManaged.sol
│   │       ├── IAccessManager.sol
│   │       └── IAuthority.sol
│   ├── crosschain
│   │   ├── CrosschainLinked.sol
│   │   └── TRC786Recipient.sol
│   └── bridges
│       ├── BridgeTRC20.sol
│       └── BridgeTRC7802.sol
```

- abstract
 - BridgeFungible.sol
- finance
 - VestingWallet.sol
 - VestingWalletCliff.sol
- governance
 - Governor.sol
 - IGovernor.sol
 - TimelockController.sol
 - extensions
 - GovernorCountingFractional.sol
 - GovernorCountingOverridable.sol
 - GovernorCountingSimple.sol
 - GovernorNoncesKeyed.sol
 - GovernorPreventLateQuorum.sol
 - GovernorProposalGuardian.sol
 - GovernorSequentialProposalId.sol
 - GovernorSettings.sol
 - GovernorStorage.sol
 - GovernorSuperQuorum.sol
 - GovernorTimelockAccess.sol
 - GovernorTimelockCompound.sol
 - GovernorTimelockControl.sol
 - GovernorVotes.sol
 - GovernorVotesQuorumFraction.sol
 - GovernorVotesSuperQuorumFraction.sol
 - utils
 - IVotes.sol
 - Votes.sol
 - VotesExtended.sol
- interfaces
 - ITRC1155.sol
 - ITRC1155MetadataURI.sol
 - ITRC1155Receiver.sol
 - ITRC1271.sol
 - ITRC1363.sol
 - ITRC1363Receiver.sol
 - ITRC1363Spender.sol
 - ITRC165.sol
 - ITRC1820Implementer.sol
 - ITRC1820Registry.sol
 - ITRC1967.sol
 - ITRC20.sol
 - ITRC20Metadata.sol
 - ITRC2309.sol
 - ITRC2612.sol
 - ITRC2981.sol
 - ITRC3156.sol
 - ITRC3156FlashBorrower.sol
 - ITRC3156FlashLender.sol
 - ITRC4626.sol
 - ITRC4906.sol
 - ITRC5267.sol
 - ITRC5313.sol
 - ITRC5805.sol
 - ITRC6372.sol

- ├─ ITRC6909.sol
- ├─ ITRC721.sol
- ├─ ITRC721Enumerable.sol
- ├─ ITRC721Metadata.sol
- ├─ ITRC721Receiver.sol
- ├─ ITRC7751.sol
- ├─ ITRC777.sol
- ├─ ITRC777Recipient.sol
- ├─ ITRC777Sender.sol
- ├─ ITRC7913.sol
- ├─ draft-IERC6093.sol
- ├─ draft-ITRC1822.sol
- ├─ draft-ITRC7674.sol
- ├─ draft-ITRC7786.sol
- ├─ draft-ITRC7802.sol
- ├─ metatx
 - ├─ TRC2771Context.sol
 - ├─ TRC2771Forwarder.sol
- ├─ proxy
 - ├─ Clones.sol
 - ├─ Proxy.sol
 - ├─ TRC1967
 - ├─ TRC1967Proxy.sol
 - ├─ TRC1967Utils.sol
 - ├─ beacon
 - ├─ BeaconProxy.sol
 - ├─ IBeacon.sol
 - ├─ UpgradeableBeacon.sol
 - ├─ transparent
 - ├─ ProxyAdmin.sol
 - ├─ TransparentUpgradeableProxy.sol
 - ├─ utils
 - ├─ Initializable.sol
 - ├─ UUPSUpgradeable.sol
- ├─ token
 - ├─ TRC1155
 - ├─ ITRC1155.sol
 - ├─ ITRC1155Receiver.sol
 - ├─ TRC1155.sol
 - ├─ extensions
 - ├─ ITRC1155MetadataURI.sol
 - ├─ TRC1155Burnable.sol
 - ├─ TRC1155Pausable.sol
 - ├─ TRC1155Supply.sol
 - ├─ TRC1155URIStorage.sol
 - ├─ utils
 - ├─ TRC1155Holder.sol
 - ├─ TRC1155Utils.sol
 - ├─ TRC20
 - ├─ ITRC20.sol
 - ├─ TRC20.sol
 - ├─ extensions
 - ├─ ITRC20Metadata.sol
 - ├─ ITRC20Permit.sol
 - ├─ TRC1363.sol
 - ├─ TRC20Burnable.sol

- TRC20Capped.sol
 - TRC20Crosschain.sol
 - TRC20FlashMint.sol
 - TRC20Pausable.sol
 - TRC20Permit.sol
 - TRC20Votes.sol
 - TRC20Wrapper.sol
 - TRC4626.sol
 - draft-TRC20Bridgeable.sol
 - draft-TRC20TemporaryApproval.sol
 - utils
 - SafeTRC20.sol
 - TRC1363Utils.sol
- TRC6909
 - TRC6909.sol
 - extensions
 - TRC6909ContentURI.sol
 - TRC6909Metadata.sol
 - TRC6909TokenSupply.sol
- TRC721
 - ITRC721.sol
 - ITRC721Receiver.sol
 - TRC721.sol
 - extensions
 - ITRC721Enumerable.sol
 - ITRC721Metadata.sol
 - TRC721Burnable.sol
 - TRC721Consecutive.sol
 - TRC721Enumerable.sol
 - TRC721Pausable.sol
 - TRC721Royalty.sol
 - TRC721URIStorage.sol
 - TRC721Votes.sol
 - TRC721Wrapper.sol
 - utils
 - TRC721Holder.sol
 - TRC721Utils.sol
- common
 - TRC2981.sol
- utils
 - Address.sol
 - Arrays.sol
 - Base58.sol
 - Base64.sol
 - Blockhash.sol
 - Bytes.sol
 - CAIP10.sol
 - CAIP2.sol
 - Calldata.sol
 - Comparators.sol
 - Context.sol
 - Create2.sol
 - Errors.sol
 - LowLevelCall.sol
 - Memory.sol
 - Multicall.sol

- └─ Nonces.sol
- └─ NoncesKeyed.sol
- └─ Packing.sol
- └─ Panic.sol
- └─ Pausable.sol
- └─ RLP.sol
- └─ ReentrancyGuard.sol
- └─ ReentrancyGuardTransient.sol
- └─ RelayedCall.sol
- └─ ShortStrings.sol
- └─ SlotDerivation.sol
- └─ StorageSlot.sol
- └─ Strings.sol
- └─ TransientSlot.sol
- └─ cryptography
 - └─ ECDSA.sol
 - └─ Hashes.sol
 - └─ MerkleProof.sol
 - └─ MessageHashUtils.sol
 - └─ P256.sol
 - └─ RSA.sol
 - └─ SignatureChecker.sol
 - └─ TIP712.sol
 - └─ TrieProof.sol
 - └─ WebAuthn.sol
 - └─ draft-TRC7739Utils.sol
 - └─ signers
 - └─ AbstractSigner.sol
 - └─ MultiSignerTRC7913.sol
 - └─ MultiSignerTRC7913Weighted.sol
 - └─ SignerECDSA.sol
 - └─ SignerEIP7702.sol
 - └─ SignerP256.sol
 - └─ SignerRSA.sol
 - └─ SignerTRC7913.sol
 - └─ SignerWebAuthn.sol
 - └─ draft-TRC7739.sol
 - └─ verifiers
 - └─ TRC7913P256Verifier.sol
 - └─ TRC7913RSABVerifier.sol
 - └─ TRC7913WebAuthnVerifier.sol
- └─ draft-InteroperableAddress.sol
- └─ introspection
 - └─ ITRC165.sol
 - └─ TRC165.sol
 - └─ TRC165Checker.sol
- └─ math
 - └─ Math.sol
 - └─ SafeCast.sol
 - └─ SignedMath.sol
- └─ structs
 - └─ Accumulators.sol
 - └─ BitMaps.sol
 - └─ Checkpoints.sol
 - └─ CircularBuffer.sol
 - └─ DoubleEndedQueue.sol


```

├── EnumerableMap.sol
├── EnumerableSet.sol
├── Heap.sol
├── MerkleTree.sol
├── types
├── Time.sol
├── vendor
├── compound
├── ICompoundTimelock.sol

```

The in-scope files of `tron-contracts-upgradeable` (84 files) were the following:

```

├── contracts
│   ├── access
│   │   ├── AccessControlUpgradeable.sol
│   │   ├── Ownable2StepUpgradeable.sol
│   │   ├── OwnableUpgradeable.sol
│   │   ├── extensions
│   │   │   ├── AccessControlDefaultAdminRulesUpgradeable.sol
│   │   │   └── AccessControlEnumerableUpgradeable.sol
│   │   └── manager
│   │       ├── AccessManagedUpgradeable.sol
│   │       └── AccessManagerUpgradeable.sol
│   ├── crosschain
│   │   ├── CrosschainLinkedUpgradeable.sol
│   │   └── bridges
│   │       ├── BridgeTRC20Upgradeable.sol
│   │       ├── BridgeTRC7802Upgradeable.sol
│   │       └── abstract
│   │           └── BridgeFungibleUpgradeable.sol
│   ├── finance
│   │   ├── VestingWalletCliffUpgradeable.sol
│   │   └── VestingWalletUpgradeable.sol
│   ├── governance
│   │   ├── GovernorUpgradeable.sol
│   │   ├── TimelockControllerUpgradeable.sol
│   │   ├── extensions
│   │   │   ├── GovernorCountingFractionalUpgradeable.sol
│   │   │   ├── GovernorCountingOverridableUpgradeable.sol
│   │   │   ├── GovernorCountingSimpleUpgradeable.sol
│   │   │   ├── GovernorNoncesKeyedUpgradeable.sol
│   │   │   ├── GovernorPreventLateQuorumUpgradeable.sol
│   │   │   ├── GovernorProposalGuardianUpgradeable.sol
│   │   │   ├── GovernorSequentialProposalIdUpgradeable.sol
│   │   │   ├── GovernorSettingsUpgradeable.sol
│   │   │   ├── GovernorStorageUpgradeable.sol
│   │   │   ├── GovernorSuperQuorumUpgradeable.sol
│   │   │   ├── GovernorTimelockAccessUpgradeable.sol
│   │   │   ├── GovernorTimelockCompoundUpgradeable.sol
│   │   │   ├── GovernorTimelockControlUpgradeable.sol
│   │   │   ├── GovernorVotesQuorumFractionUpgradeable.sol
│   │   │   ├── GovernorVotesSuperQuorumFractionUpgradeable.sol
│   │   │   └── GovernorVotesUpgradeable.sol
│   │   └── utils
│   │       └── VotesExtendedUpgradeable.sol

```

- └─ VotesUpgradeable.sol
- └─ metatx
 - └─ TRC2771ContextUpgradeable.sol
 - └─ TRC2771ForwarderUpgradeable.sol
- └─ proxy
 - └─ utils
 - └─ Initializable.sol
 - └─ UUPSUpgradeable.sol
- └─ token
 - └─ TRC1155
 - └─ TRC1155Upgradeable.sol
 - └─ extensions
 - └─ TRC1155BurnableUpgradeable.sol
 - └─ TRC1155PausableUpgradeable.sol
 - └─ TRC1155SupplyUpgradeable.sol
 - └─ TRC1155URIStorageUpgradeable.sol
 - └─ TRC20
 - └─ TRC20Upgradeable.sol
 - └─ extensions
 - └─ TRC1363Upgradeable.sol
 - └─ TRC20BurnableUpgradeable.sol
 - └─ TRC20CappedUpgradeable.sol
 - └─ TRC20CrosschainUpgradeable.sol
 - └─ TRC20FlashMintUpgradeable.sol
 - └─ TRC20PausableUpgradeable.sol
 - └─ TRC20PermitUpgradeable.sol
 - └─ TRC20VotesUpgradeable.sol
 - └─ TRC20WrapperUpgradeable.sol
 - └─ TRC4626Upgradeable.sol
 - └─ draft-TRC20BridgeableUpgradeable.sol
 - └─ draft-TRC20TemporaryApprovalUpgradeable.sol
 - └─ TRC6909
 - └─ TRC6909Upgradeable.sol
 - └─ extensions
 - └─ TRC6909ContentURIUpgradeable.sol
 - └─ TRC6909MetadataUpgradeable.sol
 - └─ TRC6909TokenSupplyUpgradeable.sol
 - └─ TRC721
 - └─ TRC721Upgradeable.sol
 - └─ extensions
 - └─ TRC721BurnableUpgradeable.sol
 - └─ TRC721ConsecutiveUpgradeable.sol
 - └─ TRC721EnumerableUpgradeable.sol
 - └─ TRC721PausableUpgradeable.sol
 - └─ TRC721RoyaltyUpgradeable.sol
 - └─ TRC721URIStorageUpgradeable.sol
 - └─ TRC721VotesUpgradeable.sol
 - └─ TRC721WrapperUpgradeable.sol
 - └─ common
 - └─ TRC2981Upgradeable.sol
- └─ utils
 - └─ ContextUpgradeable.sol
 - └─ MulticallUpgradeable.sol
 - └─ NoncesKeyedUpgradeable.sol
 - └─ NoncesUpgradeable.sol
 - └─ PausableUpgradeable.sol

```

├─ cryptography
│  └─ TIP712Upgradeable.sol
│     └─ signers
│        ├── MultiSignerTRC7913Upgradeable.sol
│        ├── MultiSignerTRC7913WeightedUpgradeable.sol
│        ├── SignerECDSAUpgradeable.sol
│        ├── SignerP256Upgradeable.sol
│        ├── SignerRSAUpgradeable.sol
│        ├── SignerTRC7913Upgradeable.sol
│        ├── SignerWebAuthnUpgradeable.sol
│        └─ draft-TRC7739Upgradeable.sol
├─ introspection
│  └─ TRC165Upgradeable.sol

```

The in-scope files of `oif-contracts` (TRON settlement adaptation) were the following:

```

├─ src
│  └─ input
│     └─ escrow
│        ├── InputSettlerEscrow.sol
│        ├── InputSettlerEscrowTron.sol
│        └─ Permit2WitnessType.sol

```

Update: The fixes for the findings highlighted in this report have all been merged at commit [d260982](#) for `tron-contracts` and at commit [8e38b9b](#) for `oif-contracts`.

System Overview

The two repositories in scope are a port of [OpenZeppelin Contracts v5.6.1](#) to the [TRON Virtual Machine \(TVM\)](#). They are general-purpose libraries of reusable building blocks: the fungible ([TRC-20](#) / [ERC-20](#)), non-fungible ([TRC-721](#) / [ERC-721](#)), multi-token ([TRC-1155](#) / [ERC-1155](#)), and minimal multi-token ([ERC-6909](#)) standards and their extensions, access control, governance, cryptography and signature utilities, proxies, meta-transactions ([TRC-2771](#) / [ERC-2771](#)), crosschain bridges, and finance helpers, rather than a deployed protocol. Downstream projects inherit and compose these contracts, so the security-relevant surface is the correctness of each component together with the points at which TVM execution differs from the EVM.

The `tron-contracts` repository is the standard library. The `tron-contracts-upgradeable` repository is the upgradeable variant of the same sources, generated for use behind proxies: its contracts use the initializer pattern in place of constructors and store state in namespaced storage locations ([TIP-7201](#) / [ERC-7201](#)) to remain upgrade-safe. Both libraries rename the standards that TRON publishes under its own [TIP](#) identifiers (for example [ERC20](#) to [TRC20](#), [EIP712](#) to [TIP712](#)), and dual-cite the TRON (TIP / TRC) and Ethereum (EIP / ERC) specifications where both exist. Several standards map to a ratified TIP, including the permit extension ([TIP-2612](#) / [ERC-2612](#)), the payable token ([TIP-1363](#) / [ERC-1363](#)), the tokenized vault ([TIP-4626](#) / [ERC-4626](#)), flash loans ([TIP-3156](#) / [ERC-3156](#)), interface detection ([TIP-165](#) / [ERC-165](#)), contract signature validation ([TIP-1271](#) / [ERC-1271](#)), proxy storage slots ([TIP-1967](#) / [ERC-1967](#)), and the minimal proxy ([TIP-1167](#) / [ERC-1167](#)). Others have no TIP yet, such as [ERC-6909](#), the [ERC-7913](#) signature verifiers, and the [ERC-7786](#) and [ERC-7802](#) crosschain interfaces.

The port adapts the parts of the library where the TVM behaves differently from the EVM. The most security-relevant adaptations are the [CREATE2 address derivation](#), which uses TRON's `0x41` prefix ([TIP-26](#)) instead of the EVM `0xff` ([EIP-1014](#)); the typed-data ([TIP-712](#) / [EIP-712](#)) domain separator, which binds to the four-byte chain identifier that TRON exposes through `eth_chainId` ([TIP-474](#)); the signed-message prefix in `MessageHashUtils`, which uses the TRON string ([TIP-191](#) / [TIP-104](#)) rather than the [ERC-191](#) one; the [TRC-721](#) receiver callback, which TRON specifies with a distinct magic value; the secp256r1 (P256) verification ([TIP-7951](#) / [EIP-7951](#)), which is performed in pure Solidity because no native precompile is active on the TVM; and a dedicated `SafeTRC20` transfer helper for [TRON USDT](#), whose `transfer` returns `false` even on a successful transfer. The [EIP-7702](#) account-abstraction module present

upstream is not included in either repository, since TRON provides multi-signature and permission features at the account level ([TIP-16](#) / [TIP-105](#)).

The `oif-contracts` repository is the [Open Intents Framework](#), a cross-chain intent settlement system in which a user locks input tokens in an escrow on a source chain so that a filler can satisfy the intent on a destination chain. In scope is its TRON adaptation, `InputSettlerEscrowTron`, which extends the framework's `InputSettlerEscrow` to run on the TVM and vendors the `tron-contracts` library as a dependency. The adaptation routes token payouts through `SafeTRC20` so that settlement works with TRON USDT. It also relies on the [Permit2](#) contract for the sponsored `openFor` collection path. Because TRON derives CREATE2 addresses with the `0x41` prefix, Permit2 is deployed at a [different address on TRON](#) than on Ethereum, so the framework's Permit2 address hook must resolve to the TRON deployment.

Security Model and Trust Assumptions

Because the repositories in scope are libraries rather than a deployed system, their security depends on how integrators configure and compose them and on the execution semantics of the target TRON network. This section records the trust assumptions relied upon during the review. Any finding whose impact depends on one of these assumptions being violated is out of scope unless an integrator or unprivileged actor could plausibly breach it.

Privileged Roles

- The libraries define no global privileged role of their own.
 - They provide the access-control and governance primitives (`Ownable`, `Ownable2Step`, `AccessControl`, `AccessManager`, `Governor`, and `TimelockController`) that integrators configure for their own deployments. The trust placed in owners, administrators, proposers, and executors is determined entirely by that configuration.

- The upgrade authority of an upgradeable deployment is fully trusted.
 - For contracts deployed behind a proxy, the proxy administrator (for transparent and beacon proxies) or the account authorized by `_authorizeUpgrade` (for UUPS proxies) can replace the implementation and therefore the entire behavior of the contract.

Trust Assumptions

- The codebase is treated as a direct port of OpenZeppelin Contracts v5.6.1.
 - No TRON-specific functionality, such as native voting or staking opcodes, is expected beyond the adaptations required to run on the TVM, and the majority of the difference from upstream is renaming rather than logic changes. Where behavior is unchanged from upstream, the corresponding upstream security properties are relied upon and are not re-derived.
- A `TRC` name is not taken to imply a ratified TRON standard.
 - Where a renamed standard maps to an existing TIP, its implementation is expected to conform to that TIP, and any deviation is reported. Where no TIP exists, mirroring the Ethereum standard is considered acceptable and is not treated as a deviation. A `TRC`-named artifact is therefore not taken to correspond to a published TRON standard.
- TRC-10 tokens are treated as unsupported.
 - The contracts implement no `TRC-10` handling, and `TRC-10 assets` are not expected to be routed through them. The review still considers whether a TRC-10 balance held by an in-scope contract could be used to create an unfavorable scenario, for example through an arbitrary call executed by a governance or timelock contract.
- TRC-20 token decimals default to 18.
 - The default `decimals()` for the TRC-20 implementation is 18 rather than TRON's native six, and is expected to be overridden where a different precision is required. Integrations pairing these tokens with six-decimal assets are expected to account for the difference.

- Tokens that return `false` on a successful transfer are handled only by the dedicated helper.
 - `TRON USDT` returns `false` from `transfer` even when the transfer succeeds. Only `SafeTRC20.safeTransferUSDT`, which verifies success via balance change rather than the return value, is relied upon for such tokens; flows that use the ordinary `safeTransfer` are not expected to be configured with them. Fee-on-transfer and rebasing tokens are treated as unsupported, as in the upstream library.
- `secp256r1` (P256) verification is performed in Solidity, without a precompile.
 - No P256 precompile ([TIP-7951](#) / [EIP-7951](#)) is active on the TVM, so verification is routed through Solidity while the function signatures are kept unchanged. The computation is more expensive than a precompile and is bounded by TRON's per-transaction execution limits.
- The energy model differs from the EVM gas model.
 - Gas-forwarding components, in particular the `TRC2771Forwarder` and the authority-based access paths, depend on gas-forwarding behavior that differs on TRON, where execution is metered as `energy` and the gas-limit and gas-price semantics are not identical to the EVM. Integrations relying on gas-limited sub-calls are relied upon to validate the behavior on TRON.
- Crosschain message delivery is delegated to a trusted external gateway.
 - The crosschain bridges delegate message delivery and replay protection to an external gateway, which is trusted to deliver each message at most once and to attribute the source chain and counterpart correctly.
- The Open Intents Framework settlement relies on an external Permit2 deployment and off-chain actors.
 - The TRON settlement contracts depend on an external Permit2 deployment for the sponsored input-collection path, expected to be configured with the correct TRON Permit2 address, and on off-chain fillers and solvers to satisfy intents. The on-chain guarantees concern only the custody and release of escrowed inputs.

Additional Considerations

- Deterministic deployment uses TRON's address derivation.
 - The `Create2`, `Clones`, and relayer helpers derive addresses with TRON's `0x41 CREATE2 prefix` ([TIP-26](#)) rather than the EVM `0xff` ([EIP-1014](#)). Off-chain tooling and any counterfactual-address computation must use the same derivation to obtain correct addresses.
- The libraries assume the target network's TVM feature gates are active.
 - Behavior that depends on a feature gate, such as transient storage ([TIP-650](#) / [EIP-1153](#)) and the optimized chain-identifier return value ([TIP-474](#)), must be confirmed against the target network, and deployments must target an EVM version the network accepts.

High Severity

H-01 TRC4626 And VestingWallet Cannot Release USDT Due To False-On-Success Transfers

Both `withdraw` and `redeem` in `TRC4626` pay the underlying asset out through a single internal function, `_transferOut`, which calls `SafeTRC20.safeTransfer`. That helper only accepts a transfer as successful when the token returns `true` or returns nothing at all.

On TRON, the `USDT` contract returns `false` from a `transfer` that has actually succeeded. `safeTransfer` reads that `false` return value, concludes the transfer failed, and reverts with `SafeTRC20FailedOperation`. Deposits are not affected, since `_transferIn` pulls assets in with `safeTransferFrom` and `USDT` does return `true` from `transferFrom`. The port already encountered this behavior and added `safeTransferUSDT` to deal with it, using it on the bridge release path in `_onReceive`, but the same change was never carried over to the vault.

This means that when the underlying is `USDT`, users can deposit and mint, but every `withdraw` and `redeem` reverts, so the shares can never be redeemed.

The same oversight is present in `VestingWallet`. Its only way to pay out a vested TRC-20 balance, `release`, also calls `SafeTRC20.safeTransfer`, so a wallet funded with `USDT` accepts the deposit but can never release it. The contract has no rescue function and is not upgradeable; transferring ownership does not help because the new owner reaches the same reverting call.

Consider paying the underlying out through `safeTransferUSDT` (or another transfer helper that confirms success from the resulting balance change) on the `TRC4626` withdrawal path, matching what `BridgeTRC20` already does. Additionally, consider applying the same fix to `VestingWallet`.

Update: Resolved in [pull request #131](#). The team stated:

`TRC4626._transferOut`, `VestingWallet.release` and `TRC20Wrapper.withdrawTo` now pay out through

`SafeTRC20.safeTransferChecked`, which confirms success from the caller's balance delta rather than the returned boolean.

- `TRC20Wrapper.withdrawTo` isn't listed in the finding but has the same defect — its underlying was trapped behind the wrapper. Fixed alongside.
- We made the checked transfer the default rather than a per-token exception, renaming `safeTransferUSDT` to `safeTransferChecked`: these contracts can't know at deployment time which token they hold, so gating on a configured USDT address would leave every other false-on-success token broken.
- The use of `safeTransferUSDT` was updated to `safeTransferChecked` on OIF at the PR: [#195](#)

H-02 Audited Contracts Inherit Issues From Upstream OpenZeppelin Contracts

The audited contracts are generated from the `tron-contracts` repository by the OpenZeppelin Upgradeability Transpiler, and `tron-contracts` is in turn based on OpenZeppelin Contracts v5.6.1. Several in-scope issues do not originate from EVM-to-TVM differences or from the transpilation, but are inherited from the upstream codebase. Most were resolved upstream after the v5.6.1 release and are scheduled for the 5.7 release, so the fixes are absent at the audited commit. In particular:

- The `baseDelaySeconds` delay of `GovernorTimelockAccessUpgradeable` can be bypassed by executing a proposal without queuing it. The `execute function` accepts proposals in both the `Succeeded` and `Queued` states, but `etaSeconds` is recorded only when `queue` is called. A proposal executed directly from the `Succeeded` state therefore has a `proposalEta` of `0`, the `block.timestamp < etaSeconds` check passes trivially, and the post-vote reaction window is skipped for operations that do not require prior scheduling. Resolved upstream in [PR #6386](#), with a follow-up in [PR #6582](#).
- The atomic mode of `TRC2771ForwarderUpgradeable` does not provide all-or-nothing execution (#30). When `refundReceiver` is the zero address, `executeBatch` reverts only on invalid requests. A valid request whose target call reverts does not abort the batch: its nonce is consumed, earlier calls remain committed, and the failed request's value is refunded through `Address.sendValue` to the zero address, permanently locking the corresponding TRX. Resolved upstream in [PR #6391](#).
- The per-target admin delay of `AccessManagerUpgradeable` can be bypassed when changing a managed contract's authority, since `setAuthority` can be scheduled and

invoked through the `execute` path, which does not apply the admin delay enforced on `updateAuthority`. Resolved upstream in [PR #6388](#).

- The `burn` and `burnBatch` functions of `TRC1155BurnableUpgradeable` perform an inline `isApprovedForAll` check instead of calling the virtual `_checkAuthorized` function, so authorization overrides that apply to transfers are silently bypassed for burns. Resolved upstream in [PR #6435](#).
- The NatSpec of the `_mintConsecutive` function of `TRC721ConsecutiveUpgradeable` states that a `batchSize` of `0` returns the number of consecutive IDs minted so far, whereas the function returns the next consecutive token ID, which differs from that count whenever `_firstConsecutiveId` is overridden to a nonzero value. This documentation error was resolved upstream in [PR #6433](#).
- The late-quorum protection of `GovernorPreventLateQuorumUpgradeable` can be bypassed when it is combined with `GovernorCountingOverridableUpgradeable` (#24). The `_tallyUpdated` function records an extended deadline only on the first quorum crossing, while `quorum is counted as the sum of For and Abstain votes` and an override vote can later move weight out of those tallies. If quorum is reached early, lost through an override, and restored immediately before the original deadline, no extension is granted and the configured reaction period is bypassed. This issue is also present upstream and has no upstream fix at the time of writing.

Consider backporting the referenced upstream fixes to the `tron-contracts` codebase, or updating the fork to version 5.7 of OpenZeppelin Contracts once it is published. Furthermore, consider reporting the late-quorum bypass upstream and addressing it in both codebases by tracking the quorum state separately from the stored extended deadline and reacting to every false-to-true quorum transition.

The items above are not necessarily exhaustive. It is advisable to review the full set of changes in the [5.7 release candidate](#) for any further upstream fixes affecting in-scope contracts that are not yet present at the audited commit.

Update: Resolved at pull request [#119](#), [#120](#), [#121](#), [#122](#), [#123](#), [#136](#), [#137](#) and [#141](#). The team stated:

We forked upstream at #6372 and reviewed all 58 subsequent commits touching contracts/. The applicable fixes are ported: #6386/#6582 (our #119), #6388→#6636 / L-12 (#120), #6391→#6415 / H-04 (#121), #6435/L-52(#122), #6433/N-32(#123), #6654(#136), #6644/M-03 + #6681 (#137), #6573/M-01(#141). H-02 (#6618) needed no change — our tree already rejects the malformed case through an equivalent guard — and M-20 (#6643) does not apply, since we ship only BridgeFungible.

Seven upstream fixes remain unported: #6642 (M-08), #6646 (L-08), #6638, #6635, #6418. Since those don't carry funds at risk or major issues, so they were scheduled for the 5.7 sync. This way we can keep both tron-contracts and openzeppelin-contracts up-to-date.

Medium Severity

M-01 InputSettlerEscrowTron Uses Ethereum Permit2 Address, Breaking openFor on TRON

The `InputSettlerEscrow` contract escrows inputs through two paths: the direct `open`, which pulls tokens via `safeTransferFrom`, and the sponsored `openFor`, which pulls a user's signed inputs through Permit2 at the address returned by `__PERMIT2`. That getter is `virtual` and defaults to the canonical Permit2 address `0x00000000000022D473030F116dDEE9F6B43aC78BA3`. Its documentation states that it must be overridden on chains where Permit2 lives elsewhere, and names TRON, whose CREATE2 derivation differs.

However, `InputSettlerEscrowTron` overrides only the payout hook `__transfer` and leaves `__PERMIT2` at the canonical address. Because TRON derives CREATE2 addresses with the `0x41` prefix, Permit2 is not deployed at that canonical address on TRON. Therefore, the Permit2 call transfers no inputs, so the sponsored `openFor` path cannot escrow funds on TRON. Although the override that resolves this was already introduced for TRON and is covered by a dedicated test, it was not applied to the settler intended for TRON deployment.

Consider overriding `__PERMIT2` in `InputSettlerEscrowTron` to return the Permit2 address on TRON. Alternatively, if `openFor` is not intended to be supported on TRON, consider documenting that limitation and removing the unused Permit2 path.

Update: Resolved in [pull request #190](#). The team stated:

The issue is solved by overriding the `__PERMIT2()` internal function on `InputSettlerEscrowTron.sol` to return the real deployment on Tron Mainnet.

Low Severity

L-01 TRC20FlashMint Rejects TIP-3156-Compliant Flash Borrowers

The `flashLoan` function of `TRC20FlashMint` mints the requested tokens to the receiver, invokes its `onFlashLoan` callback, and requires the callback to return a fixed magic value confirming that the receiver is a willing flash borrower. This magic value is the `RETURN_VALUE` constant, which is computed as `keccak256("ERC3156FlashBorrower.onFlashLoan")`, the Ethereum preimage carried over from upstream. However, [TIP-3156](#), the TRON counterpart the port targets, requires borrowers to return `keccak256("TRC3156FlashBorrower.onFlashLoan")` instead. As a result, a TIP-3156-compliant borrower returns a value that never matches `RETURN_VALUE`, so `flashLoan` reverts with `TRC3156InvalidReceiver`, and the feature remains interoperable only with ERC-3156-style borrowers rather than the TRON-native borrowers it is meant to serve.

Consider deriving the return value from `"TRC3156FlashBorrower.onFlashLoan"` and updating the `ITRC3156FlashBorrower` documentation, confirming the value against the formal specification since TIP-3156 is in Last Call.

Update: Resolved in [pull request #114](#). The team stated:

| *We have changed the return value and documentation to follow TIP 3156.*

L-02 Documentation Cites Ethereum ERC/EIP Names for Standards Renamed to TRON Identifiers

The library renames the standards it implements to their TRON identifiers such as `TRC20`, `TRC1155`, and `TIP712`, and in most places documents them under the TRON name. In a number of files, however, the documentation still refers to a renamed standard only by its Ethereum name, even though the port cites the TRON equivalent elsewhere. These references are documentary and do not affect on-chain behavior, but they contradict the naming the port adopts everywhere else.

The following instances were identified:

- The [proxy README](#) describes storage slots as "ERC-1967" and titles a section `ERC-1967`, while the port ships `ITRC1967`.
- [IGovernor](#) refers to the "EIP-712 domain separator", although the port renames EIP-712 to TIP-712 and cites `TIP-712` throughout the rest of the codebase.
- The [cryptography README](#) mentions "ERC-1271 signatures", while the port ships `ITRC1271`.
- The [utils README](#) mentions "ERC-7201 namespaces", which should be TIP-7201.
- [MessageHashUtils](#) points to "EIP-712" and "ERC-5267".
- The interfaces [ITRC3156FlashBorrower.sol](#), [ITRC3156FlashLender.sol](#), [ITRC1363.sol](#), [ITRC1363Receiver.sol](#), [ITRC1363Spender.sol](#) and [ITRC1820Registry.sol](#) only cite `ERC` instead of `TRC` or dual-citation in the comments.
- The contracts [TRC1363Utils.sol](#), [TRC20FlashMint.sol](#), [Clones.sol](#), [UUPSUpgradeable.sol](#) and [TRC165Mock.sol](#) cite `ERC` instead of `TRC` or dual-citation in the comments.
- The documentation at [contracts/proxy/README.adoc](#), [contracts/token/TRC20/README.adoc](#), [contracts/utils/README.adoc](#), and [contracts/utils/cryptography/README.adoc](#) cite `ERC` instead of `TRC` or dual-citation in the commit.
- The function name - `erc7201Slot` can be renamed to `trc7201Slot`.

Not every Ethereum reference is a mistake, and a mechanical find-and-replace would introduce errors. Three categories are correct as written and should be left in place:

- References that already dual-cite the TRON standard alongside the Ethereum original, such as [Initializable](#), which documents its namespace as "TIP-7201 (the TRON-side analogue of ERC-7201)".
- Standards that have no current TIP including the [ERC-7786](#), [ERC-7802](#), [ERC-7913](#), for which the Ethereum name is the only accurate reference.
- Ethereum-specific concepts with no TRON analogue, such as [ERC-4337](#), [EIP-7702](#), [EIP-155](#), and [EIP-170](#), together with the callback names deliberately retained per their TRON standards ([onERC1155Received](#) and [onERC721Received](#)).

Consider updating the instances listed above to cite the TRON identifier of each renamed standard and only citing the Ethereum original when necessary.

Update: Resolved in pull request [#125](#) and [#135](#). The team stated:

We have addressed the issues. All documentation instances now use the TRON identifier; the `erc7201Slot` → `trc7201Slot` change is in a separate PR #125. Four instances: `ITRC3156FlashBorrower`, `ITRC3156FlashLender`, `TRC20FlashMint`, and the `TRC20FlashMint` entry in `token/TRC20/README.adoc` are fixed in the same PR #125.

The convention has been applied: the port uses the TRC/TIP identifier for every standard it re-ships, since that is what the shipped artifacts are named (`ITRC1820Registry`, `ITRC2309`, `BridgeTRC7802`). Where TRON has published a TIP, the citation links it and notes the Ethereum analogue: `.../tip-1363.md[TIP-1363]` (the TRON-side analogue of `...[EIP-1363]`). Where TRON has not, the identifier still reads TRC-N but the citation points at the document that defines it: `TRC-7751` (see `...[ERC-7751]`). No link is ever labelled TRC while targeting an Ethereum document.

L-03 Missing Documentation for Decoding 21-Byte TRON Addresses

Inside the TVM an account is 20 bytes, but TRON's canonical external address form is the 21-byte `0x41`-prefixed encoding that every wallet, explorer, and most tooling display. Several contracts in scope decode or validate an address from raw bytes carried in a cross-chain payload, yet none document which of the two forms a caller or relayer must supply. When the two forms are confused the intended flow breaks, and the failure mode differs by component.

On the receive path of the fungible bridge,

`BridgeFungibleUpgradeable._processMessage` and its non-upgradeable counterpart `BridgeFungible._processMessage` decode the destination as `address to = address(bytes20(toBinary))` with no check that `toBinary` is exactly 20 bytes, and the send path forwards the recipient bytes from `parseV1()` without a length bound. If a counterpart bridge or relayer encodes the recipient in the natural 21-byte form, `bytes20()` truncates it to `0x41` followed by the first 19 bytes, a different address, and the release transfers funds there without reverting. This requires only an honest tooling error, not an attacker, and results in permanent loss. For the mint bridge, `BridgeTRC7802`, it mints new supply to the wrong address with no clawback.

The Open Intents Framework exhibits the same 21-byte ambiguity with the opposite failure mode. Its `LibAddress.validatedCleanAddress` converts a cross-chain identifier to an address only after requiring that the upper twelve bytes are zero, reverting with `HasDirtyBits` otherwise. A TRON recipient encoded in its 21-byte `0x41`-prefixed form places the `0x41` byte above the low twenty bytes, so `identifier >> 160` is non-zero and the call reverts. This fails safe rather than losing funds. However, it blocks settlement whenever

an identifier carries a TRON address in its natural form, which can prevent the framework from integrating on TRON.

The common cause is that these byte-manipulation paths assume the 20-byte TVM form without validating the input length or documenting the requirement.

Consider thoroughly documenting the handling of byte manipulation on 21-byte TRON input addresses across the codebase, stating explicitly that an address input must be the 20-byte TVM account and not the 21-byte `0x41` form, and making the expected encoding clear to counterpart bridges, relayers, and off-chain tooling. Consider also adding explicit length validation on the bridge paths, for example `require(toBinary.length == 20)` in `_processMessage` and an equivalent `addr.length == 20` check on the send path, so that a mismatched input reverts instead of being silently truncated. Since `LibAddress.validatedCleanAddress` already rejects the 21-byte form, consider ensuring the documentation for the Open Intents Framework makes clear that integrators must supply the 20-byte encoding.

Update: Resolved in pull request [#117](#) and [#136](#) in `tron-contracts` repo and pull request [#196](#) in `oif-contracts` repo. The team stated:

It is true that Tron shows addresses as 21 bytes in their wallets, but this is simply an UI choice and doesn't have any onchain implication. Developers must be aware of how to translate the Base58 21 bytes addresses to the actual 20 bytes addresses at the smart contract level.

However, adding a custom error `BridgeInvalidRecipient` is positive for the library and improves security so it was added on <https://github.com/OpenZeppelin/tron-contracts/pull/117>

We have updated the documentation for `tron-contracts-upgradeable` and `oif-contracts` repos.

Notes & Additional Information

N-01 TRON Block Time Compresses Block-Denominated Governance Windows Roughly Fourfold

The Governor and Votes stack is byte-identical to the audited OpenZeppelin Contracts v5.6.1 and is clock-agnostic, so no code defect exists. However, the default clock is block-number based: `Votes.clock` returns `Time.blockNumber` and `CLOCK_MODE` is `mode=blocknumber&from=default`. TRON produces blocks roughly four times faster than Ethereum, approximately every 3 seconds against approximately every 12 seconds, so every block-denominated window elapses in about a quarter of the intended wall-clock time when a deployer reuses Ethereum-style block-count parameters.

Block-denominated parameters affected include `votingDelay`, `votingPeriod`, the proposal snapshot and deadline, all vote and quorum checkpoints, and the `voteExtension` of `GovernorPreventLateQuorum`. Timestamp-denominated values are unaffected, including all timelock delays, `VestingWallet` and its cliff variant, and all signature expiries such as `permit` and `delegateBySig`. Two shipped artifacts bake in the 12-second assumption: the governance documentation at `docs/modules/R00T/pages/governance.adoc` states `votingDelay = 1 day = 7200 blocks` and `votingPeriod = 1 week = 50400 blocks`, which on TRON correspond to roughly 6 hours and 1.75 days; and the `MyGovernorUpgradeable` and `MyGovernor` mock returns `7200` and `50400` with day/week comments while its paired token uses the default block clock.

Consider correcting the block-time arithmetic in the documentation and mock for TRON (approximately 28800 blocks per day and 201600 blocks per week at 3 seconds) and adding TRON-specific governance deployment guidance. Deployers should preferably be steered toward the timestamp clock mode, as demonstrated by `MyTokenTimestampBasedUpgradeable`, so that windows are expressed in seconds and independent of block time; when the block clock is used, Ethereum block counts should be multiplied by approximately four and `lateQuorumVoteExtension` re-checked.

Update: Resolved in [pull request #116](#). The team stated:

| *We implemented the fix on docs and mock contract.*

N-02 Stale Warnings And Cautions In The Codebase

Throughout the codebase, the following cautions and warnings were identified in documentation that can be considered stale or misleading given the migration to the TRON ecosystem:

1. The proxy module documentation in [contracts/proxy/README.adoc](#) recommends the OpenZeppelin Upgrades Plugins for Hardhat and Foundry as the default way to deploy and manage upgradeable proxies:
 - **CAUTION:** Using upgradeable proxies correctly and securely is a difficult task that requires deep knowledge of the proxy pattern, Solidity, and the EVM. Unless you want a lot of low level control, we recommend using the OpenZeppelin Upgrades Plugins for Hardhat and Foundry.
 - The recommended plugins are Ethereum-oriented. They depend on `eth_*` JSON-RPC methods and default provider/compiler behavior that are not guaranteed to match a TRON/TVM environment, where deployments and upgrades are sensitive to (i) which RPC methods and block tags the provider supports, and (ii) the bytecode produced by the Solidity compilation target (`evmVersion` / opcode set). Following this recommendation unmodified on TRON can result in failed deployments or upgrades, partially completed deployments, or proxies that cannot be upgraded in the target environment without redeploying.
2. The **WARNING** in [TIP712Upgradeable](#) (`__TIP712_init`) describes a `_nameFallback` / `_versionFallback` mechanism, an immutable `_hashedName` / `_hashedVersion` cache, and a "keep name/version within 31 bytes behind a proxy or clone" rule; none of these exist in this implementation, which stores plain `_name` / `_version` strings of any length in ERC-7201 storage read identically by `_buildDomainSeparator` and `eip712Domain`, so the warned desynchronization cannot occur.
3. The **NOTE** in [draft-TRC7739Upgradeable](#) claims a ShortStrings 31-character optimization and an ERC-4337/ERC-7562 storage-access limitation; no ShortStrings path exists, and the domain reads during ERC-7739 validation touch the account's own ERC-7201 slots, which ERC-7562 permits for the sender. Finally, the comment in [MultiSignerTRC7913WeightedUpgradeable](#) states that `_totalExtraWeight` is packed with the base contract's `_threshold`, but the two fields reside in separate ERC-7201 namespaces and cannot share a storage slot.

Consider updating the above documentation to avoid misinformation and confusion.

Update: Resolved in pull request [#132](#) and [#134](#).

N-03 Rename contracts from OpenZeppelin Contracts to Tron Contracts

The comment at the top of each contract file currently uses the text `// OpenZeppelin Contracts (last updated v5.6.0) (governance/TimelockController.sol)`.

For consistency within the repository, this should be updated to

`// Tron Contracts (last updated v5.6.0) (governance/TimelockController.sol)` in all similar instances.

Consider updating all such header comments to reference `Tron Contracts` instead of `OpenZeppelin Contracts` to maintain consistent naming throughout the codebase.

Update: Resolved in [pull request #124](#). The team stated:

| *We have updated the comments and scripts.*

N-04 Governance Executors Do Not Support Forwarding or Recovery of TRC-10 Tokens

TRON contracts can hold native TRC-10 assets independently of TRX. The TVM exposes their incoming amount and identifier through `msg.tokenvalue` and `msg.tokenid`, and forwarding them requires the token-aware `CALLTOKEN` operation or Solidity's `address.transferToken` primitive. The [TVM opcode reference](#) identifies `CALLTOKEN` as the operation that invokes a contract with TRC-10, and an official pinned TRON fixture demonstrates [address.transferToken\(amount, id\)](#) and the incoming token context. The `Governor` and `TimelockController` executors expose payable entry points through which a TVM caller can attach TRC-10, and their payable receive paths can hold native assets as well.

Every supplied outbound path nevertheless uses an ordinary Solidity call that carries only TRX. The `Governor` performs the call in `_executeOperations` and `relay`, while the `TimelockController` centralizes the same operation in `_execute`. The proposal and operation hashes bind targets, TRX values, and calldata, but contain no TRC-10 token identifier or amount. Therefore, governance cannot attach TRC-10 to an executed target call, and native tokens credited to either executor cannot be recovered through the supplied

execution or relay paths. This is particularly surprising for `relay`, whose documentation advertises recovery of mistakenly sent tokens or TRX; ordinary TRC-20 tokens are recoverable by calling their contract, but native TRC-10 tokens are not. No attacker profit path was identified, so the impact is limited to accidental or intentional asset unavailability.

The behavior is present identically in both libraries. In `tron-contracts`, the payable `Governor.execute`, `Governor.relay`, `TimelockController.execute`, and `TimelockController.executeBatch` entry points contrast with the TRX-only outbound calls in `Governor._executeOperations` and `TimelockController._execute`. The same holds in `tron-contracts-upgradeable`.

Consider adding a self-governed TRC-10 recovery operation using `address.transferToken`. If arbitrary token-bearing governance calls are intended, add token-aware execution variants and bind both the token identifier and the token value into the proposal or operation hash. Otherwise, document explicitly that TRC-10 is unsupported. Rejecting a nonzero `msg.tokenvalue` on the payable entry points can prevent call-attached deposits, although it cannot necessarily prevent every protocol-level transfer.

Update: Resolved in pull request [#130](#) and [#133](#). The team stated:

The library is not intended and does not support TRC-10 tokens. The Tron team does not incentivize its use and the default for tokens is TRC-20. PR created to document it

N-05 README and Documentation Reference ERC/EIP-Named Artifacts That Only Exist Under TRC Names

Several references in the user-facing documentation of both TRON libraries point to `ERC` / `EIP`-named artifacts (contracts, functions, and a documentation page) that do not exist under those names in the port; the correctly named `TRC` artifact exists and is what the documentation should reference. These are documentation defects with no on-chain impact: one is a broken link, and the others direct integrators to nonexistent contract and function names.

The affected artifact references are the following:

- The upgrades guide instructs deploying an `ERC1967Proxy`, but the package ships `TRC1967Proxy`, not `ERC1967Proxy`.

- The signature-checking example calls `SignatureChecker.isValidERC1271SignatureNow`, whereas `SignatureChecker` exposes `isValidTRC1271SignatureNow`.
- The introspection section references `IERC165`, `ERC165`, and `ERC165Checker` (including `using ERC165Checker for address;`), whereas the port provides `ITRC165`, `TRC165`, and `TRC165Checker`.
- A governance paragraph uses a lone no-dash `TRC6372 clock` in text that otherwise refers to the standards as `ERC-6372` and `ERC-5805`.
- The contribution guidelines require error-name domain prefixes of the form `ERC<number>`, whereas this codebase standardizes on `TRC<number>`.

With one exception, these appear in both libraries: in `tron-contracts`, in its [upgrades guide](#), [utilities guide](#), [governance guide](#), and [GUIDELINES.md](#); and identically in `tron-contracts-upgradeable`, in its [upgrades guide](#), [utilities guide](#), [governance guide](#), and [GUIDELINES.md](#).

The exception is the broken documentation-page link, which appears only in `tron-contracts-upgradeable`: its [README.md](#) links a nonexistent `erc6909.adoc` page, whereas the actual page is `trc6909.adoc`, and it lists "ERC-6909". The `tron-contracts README.md` contains no equivalent reference, so this item does not apply to it.

Consider a single documentation pass across both libraries that repoints the `ERC / EIP`-named artifact references to their `TRC` equivalents, and, in `tron-contracts-upgradeable`, fixes the broken `erc6909.adoc` link. Dash-form references to the underlying standards (for example `EIP-712` or `ERC-165` as a specification) are correct and should be left unchanged.

Update: Resolved in [pull request #126](#). The team stated:

The `erc6909.adoc` link is reported as appearing only in `tron-contracts-upgradeable` because the README on the upgradeable library is generated through an script, which was also fixed in the PR.

Conclusion

The `tron-contracts` and `tron-contracts-upgradeable` libraries port OpenZeppelin Contracts to the TRON Virtual Machine, and a third repository adapts the Open Intents Framework settlement contracts to TRON. OpenZeppelin audited the three codebases and the adaptations required to run on the TVM.

Most of the change from upstream is renaming, and the reported findings fall into two areas. The first is where TVM execution diverges from the EVM: TRON USDT returning `false` on a successful transfer freezes `TRC4626` withdrawals, the 21-byte TRON address form is mishandled when decoding recipients, the settlement layer must target TRON's own Permit2 deployment, and TRON's faster block time compresses block-denominated governance windows. The second is standards and documentation conformance, including residual Ethereum `ERC / EIP` names that should reference the TRON identifiers and a flash-loan callback value that does not match the TRON standard.

The most significant cross-cutting concern is that the port is pinned to a fixed upstream baseline and does not carry over fixes that OpenZeppelin has since made upstream. Several issues already resolved upstream remain present, so a process to track upstream advisories and backport their fixes should be established before the libraries are relied upon in production.

OpenZeppelin thanks the development team for their responsiveness and for the context provided throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.